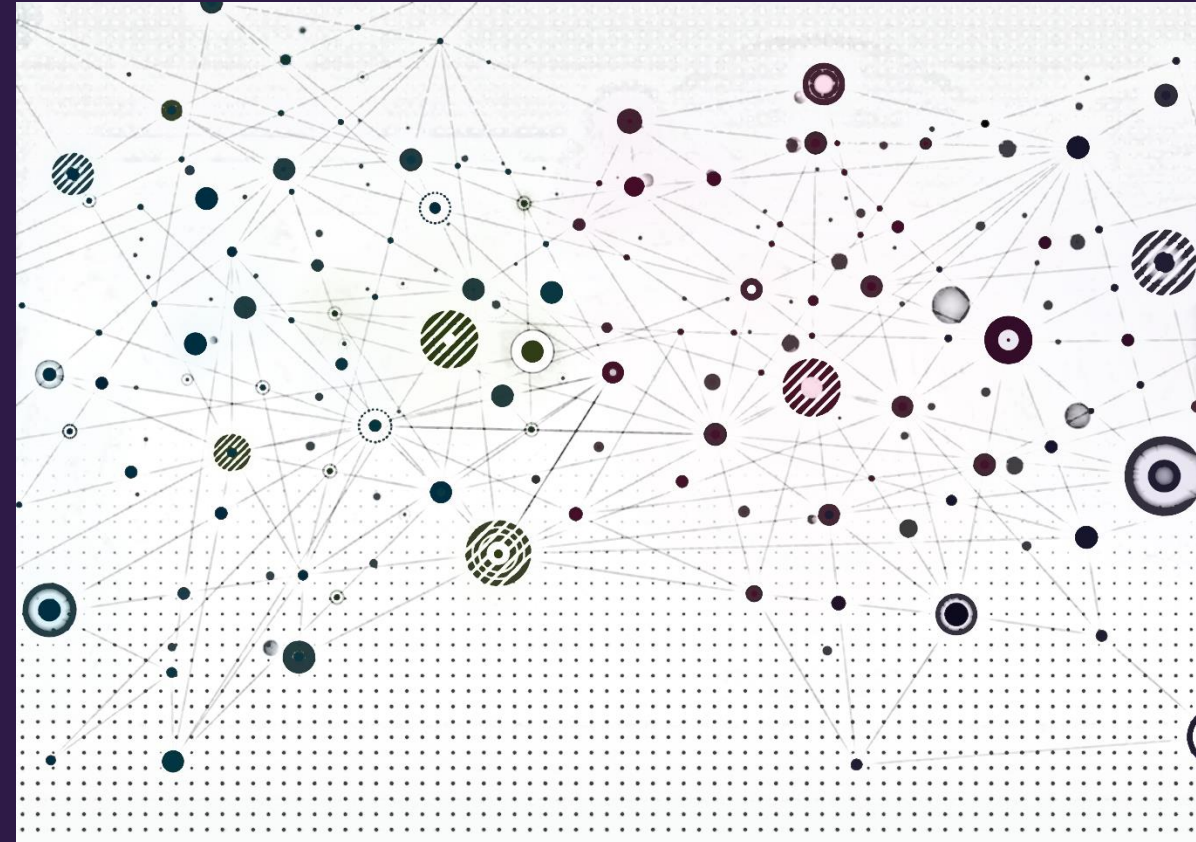


GNNs for Recommender Systems





Overview

- Introduction to recommender systems
- Neural graph collaborative filtering (NGCF)
- LightGCN
- Model training



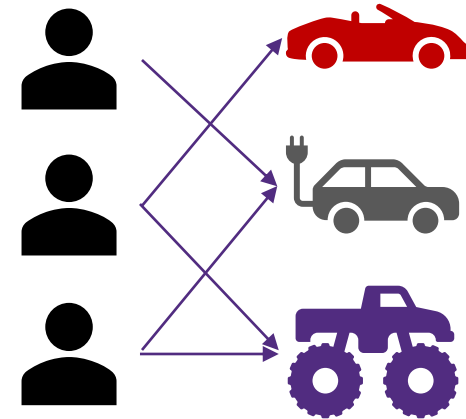
Introduction to recommender systems





What is a recommendation system?

- Typically, includes users and items
- Items can be anything: media content, clothing, websites, medications, ...
- Goal is to recommend new items to users that they are likely to "consume" (i.e., watch, read, buy, click on, ...)
- Approaches generally seek to:
 - Identify users similar to current user and recommend items consumed by the similar users
 - And/OR: identify similar items to those the current user has previously "consumed"





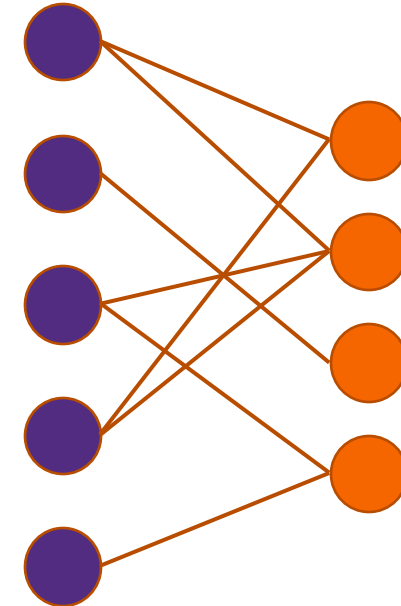
Why do we need recommendation systems?

- Information deluge and “choice” overload
 - >10K movies on Netflix
 - >12M products on Amazon
 - >70M music tracks on Spotify
 - >10B videos on YouTube
 - >200B pins on Pinterest
- Personalized recommendation (i.e., suggesting a small number of items that are likely to be interesting to the user) is critical



Bipartite graph

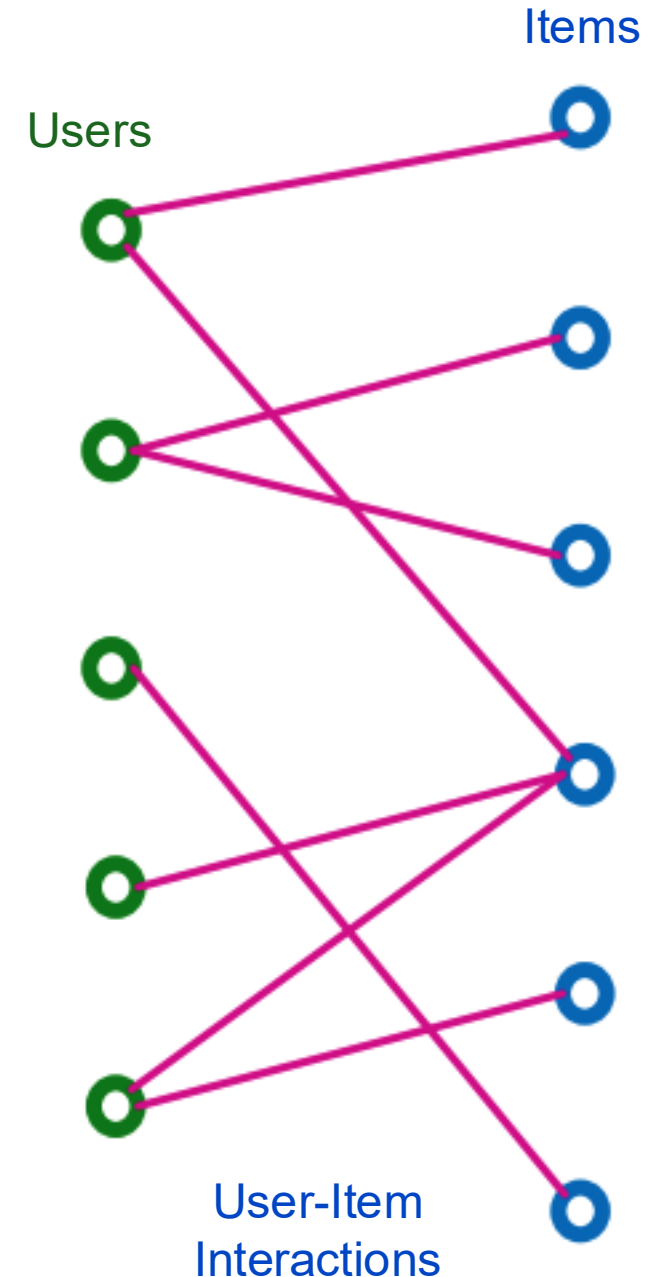
- A graph in which the nodes, $\mathcal{V}$, can be divided into two groups $\mathcal{V}_1$ and $\mathcal{V}_2$ such that every edge connects a node in $\mathcal{V}_1$ to a node in $\mathcal{V}_2$
- Special type of heterogeneous graph with two (2) node types and one (1) relation type



All edges in a bipartite graph are between nodes of two distinct types

Recommender system as a graph

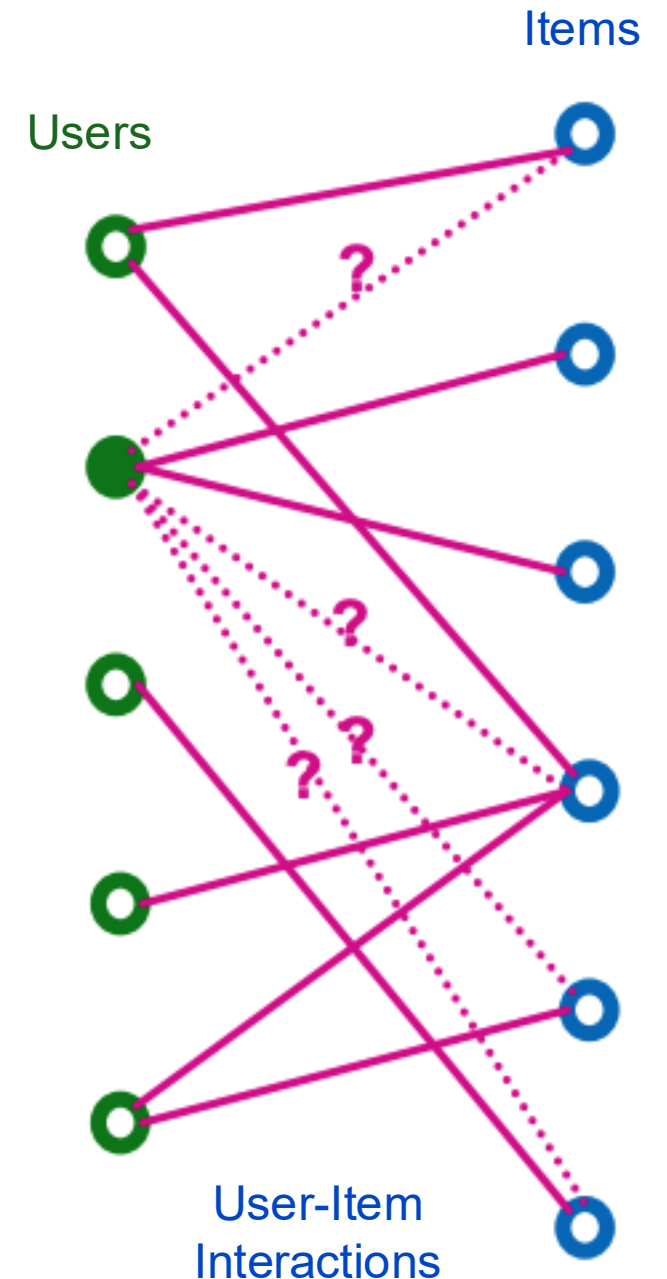
- A recommender system can be modelled as a bipartite graph
- Node types
 - Users
 - Items
- Edges connect users to items
 - Indicates a user-item interaction (e.g., purchase, view, etc)
 - Typically associated with a timestamp





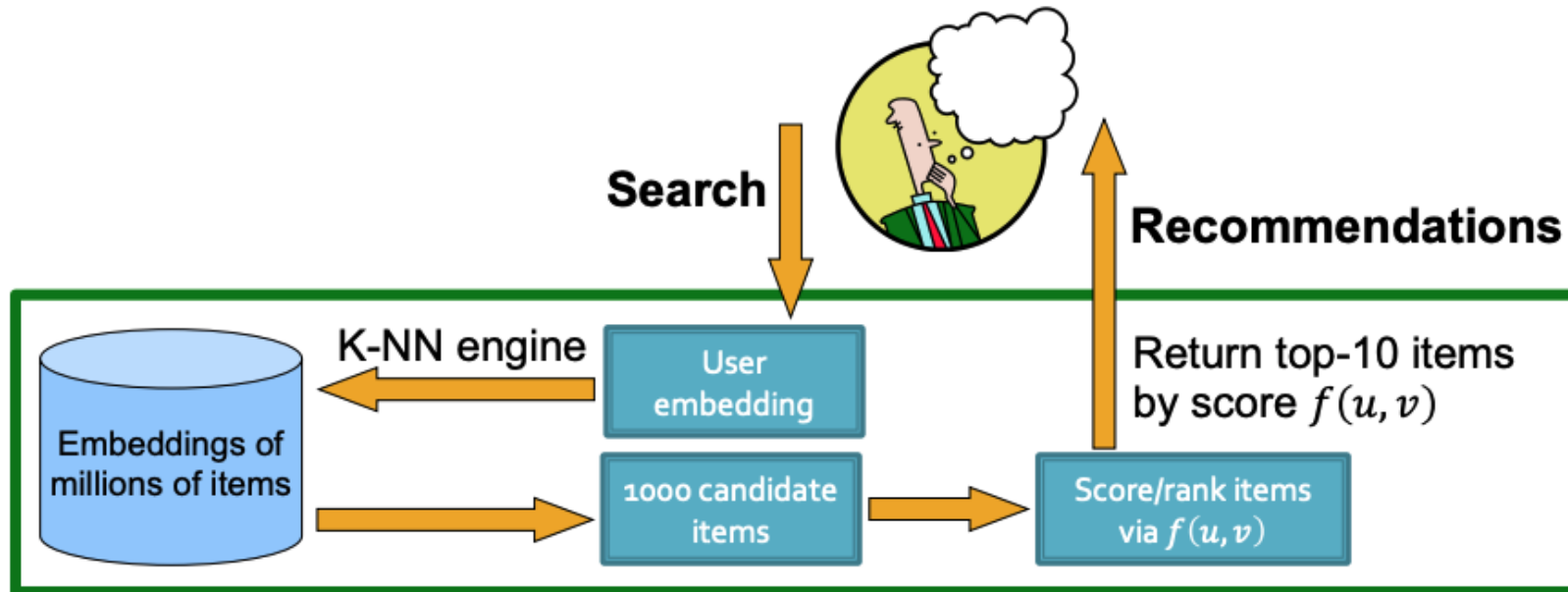
Recommendation task

- Given: past user-item interactions
- Task
 - Predict new items with which the user will interact
 - For $u \in \mathbf{U}$ (users) and $v \in \mathbf{V}$ (items), need a real-valued score $f(u, v)$
 - Can be cast as a link prediction problem – predict new user-item interaction edges given past edges



Recommender systems at scale

- Problem: For systems with large numbers of users and items, cannot efficiently evaluate $f(u, v)$ for every user-item pair
- Practical implementation: 2-stage process
 - Generate candidate solutions
 - Rank the solutions and recommend the top few





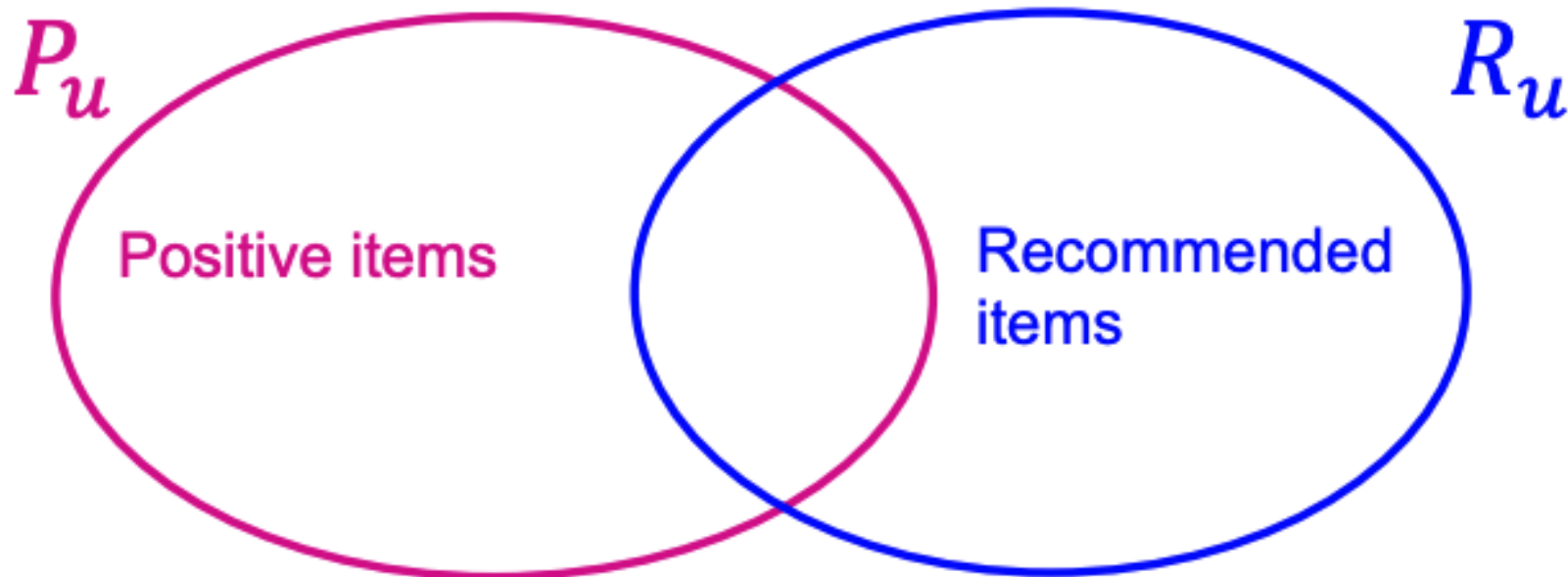
Top- K recommendations

- For each user, recommend K items
 - To be effective, K needs to be much smaller than the total number of items available
 - K is typically 10– 100
- The goal is to include as many **positive items** as possible in the top- K recommended items where
 - **Positive item** → one with which the user will interact
 - **Evaluation metric**: Recall@ K (a.k.a. Hits@ K)



Recall@K metric

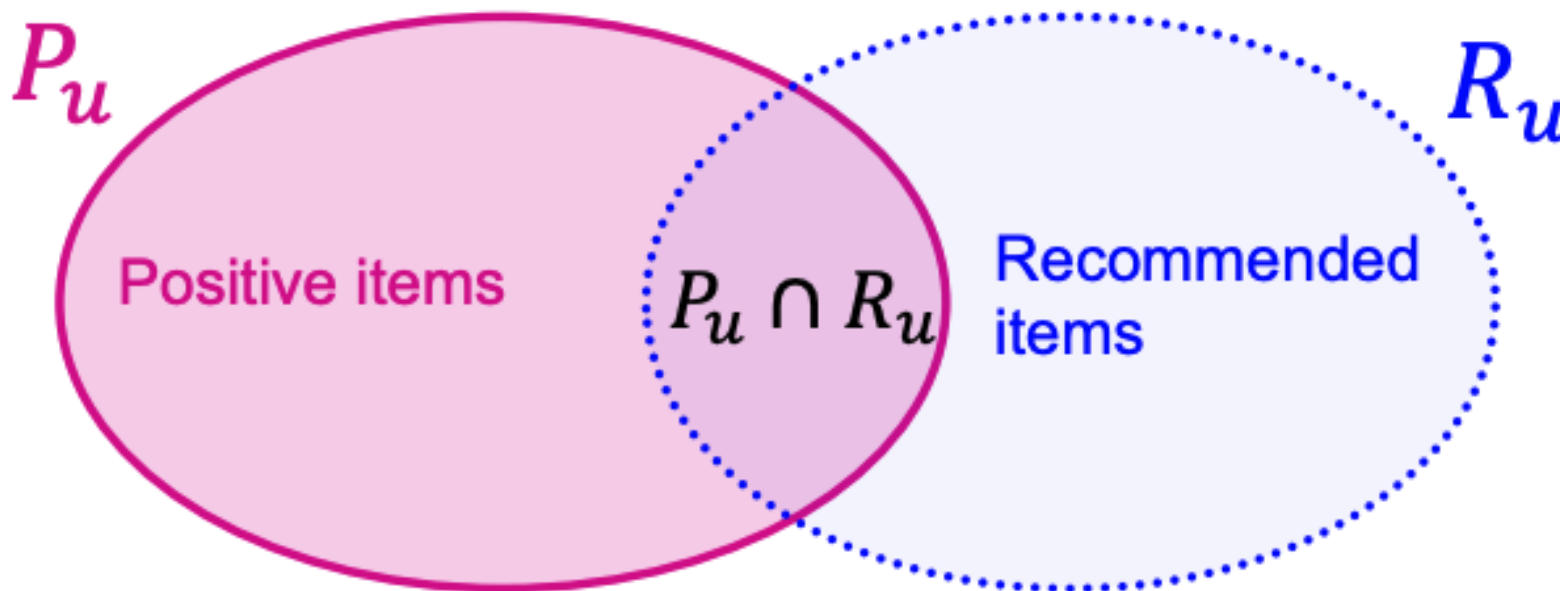
- For a given user, u
 - Let P_u be the set of positive items with which the user will interact
 - Let R_u be the set of items recommended by the system
 - In top- K approach, $|R_u| = K$
 - Items with which the user has previously interacted are excluded





Recall@K metric

- Recall@K for user u is $\frac{|P_u \cap R_u|}{|P_u|}$
 - Higher value indicates more positive items are in recommended in top-K for user u
 - The final **Recall@K for the system** is the average of the values across all users





Notation

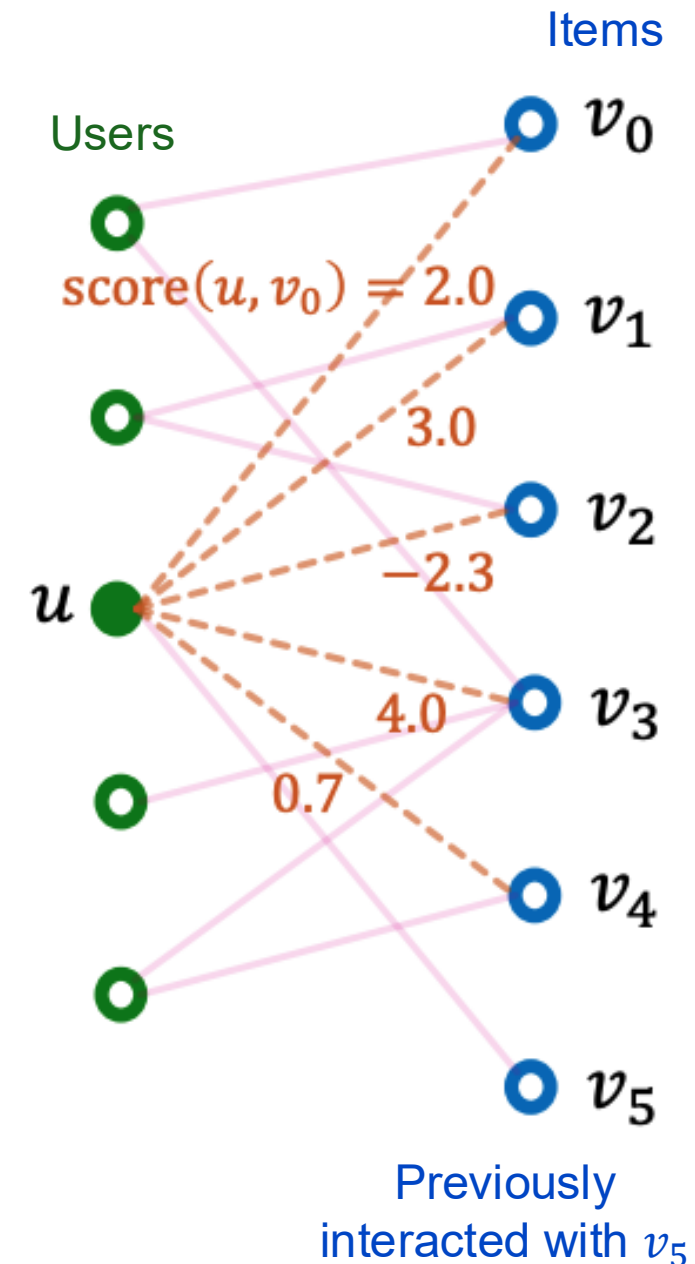
- **U**: set of all users (entities / nodes)
- **V**: set of all items (entities / nodes)
- **E**: set of all user-item interactions (edges)

$$\mathbf{E} = \{(u, v) | u \in \mathbf{U}, v \in \mathbf{V}\}$$



Score function

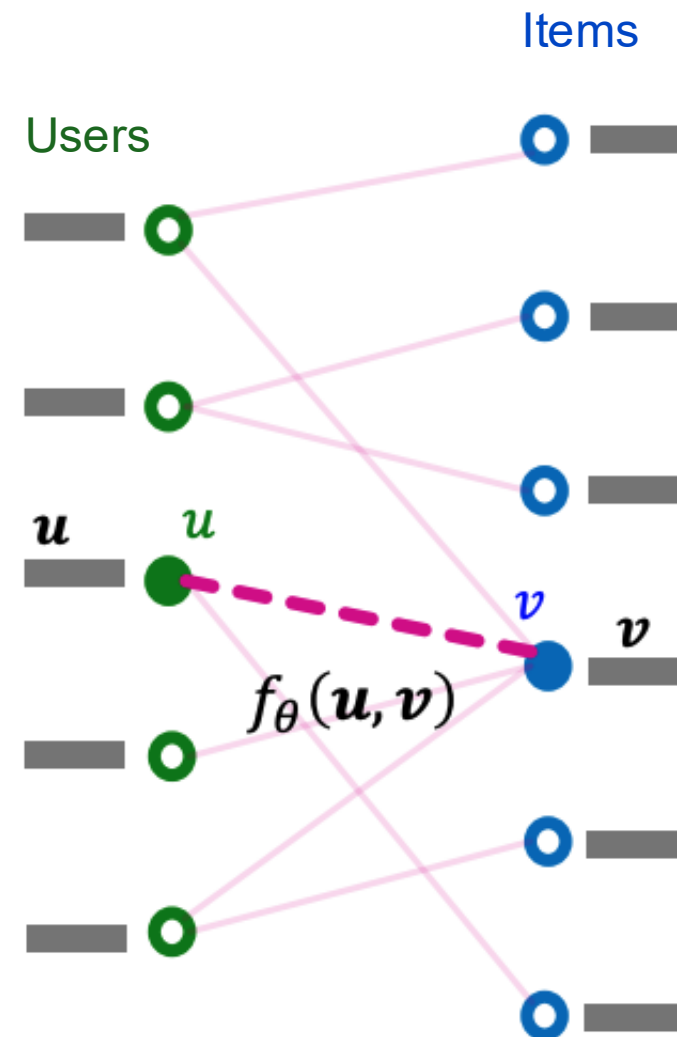
- To get the top- K items, we need a score function for each user-item interaction, that is:
 - For $u \in \mathbf{U}, v \in \mathbf{V}$ we need a real-valued scalar $score(u, v)$
 - K items with the highest scores for user, u



Embedding based models

- We'll consider **embedding-based models** to represent users and items and score interactions
 - Let $\mathbf{u} \in \mathbb{R}^d$ be the embedding for user $u \in \mathbf{U}$
 - Let $\mathbf{v} \in \mathbb{R}^d$ be the embedding for user $u \in \mathbf{V}$
 - Let $f_{\theta}(\mathbf{u}, \mathbf{v}): \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ be a parameterized function such that

$$\text{score}(u, v) = f_{\theta}(\mathbf{u}, \mathbf{v})$$
- Training objective:** optimize model parameters to achieve "high" recall@K for previously observed (i.e., supervised training) user-item interactions





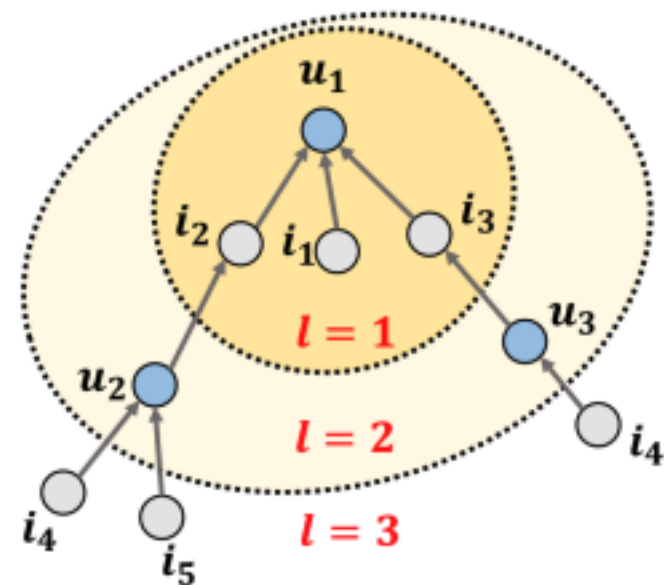
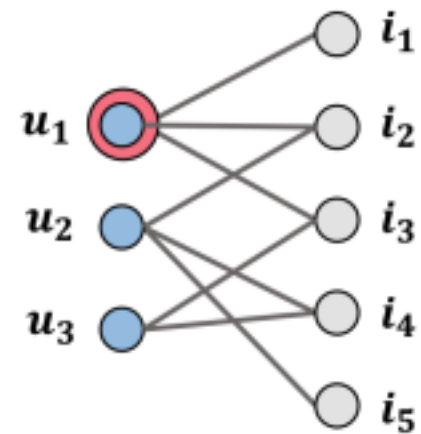
Neural graph collaborative filtering





Neural graph collaborative filtering (NGCF)

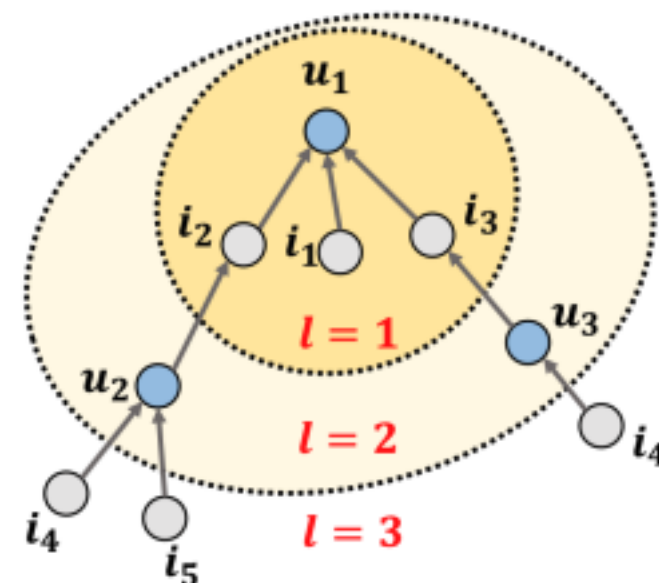
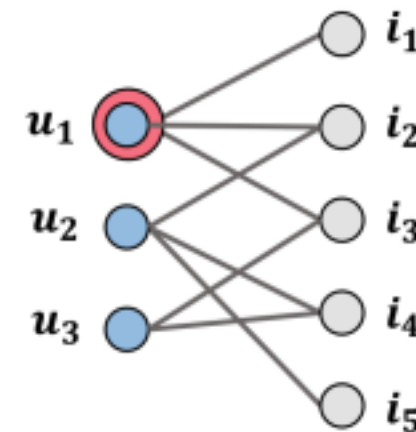
- Incorporate high-order graph structure when generating user / item embeddings
- Use a GNN to generate user / item embeddings





Neural graph collaborative filtering (NGCF)

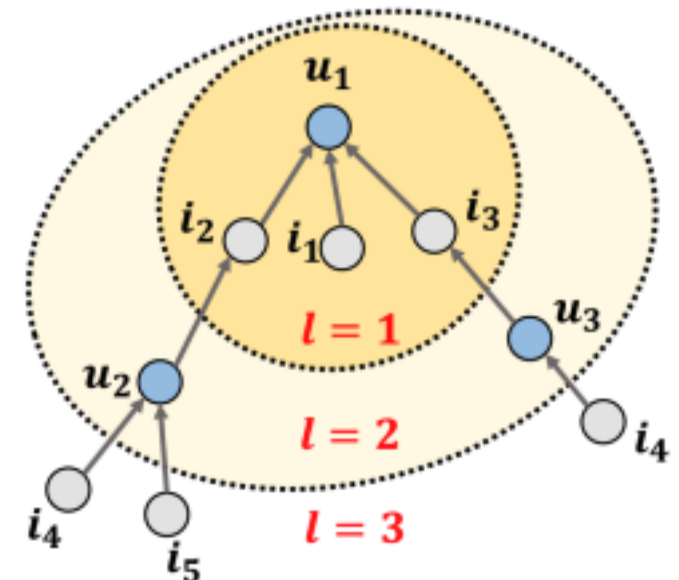
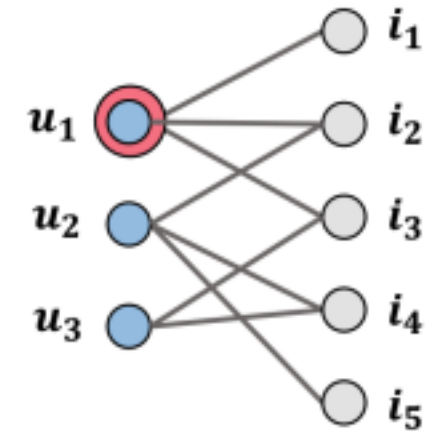
- Given: user-item bipartite graph
- NGCF framework
 - Randomly initialize embeddings for each user & item node
 - Use multi-layer GNN to propagate messages along the bipartite graph to update embeddings
 - Final embeddings are explicitly aware of the graph structure (higher order interactions of users & items)
- Two types of learnable parameters
 - Initial user / item embeddings (this is different than what we've done previously)
 - GNN parameters





Neural graph collaborative filtering (NGCF)

- Set the initial user / item embeddings For every user $u \in \mathbf{U}$ set a random vector $\mathbf{h}_u^{(0)}$ as the user's initial embedding (features)
 - For every user $v \in \mathbf{U}$ set a random vector $\mathbf{h}_v^{(0)}$ as the user's initial embedding (features)
 - These are treated as trainable parameters that will be learned (this is different than what we've done previously)





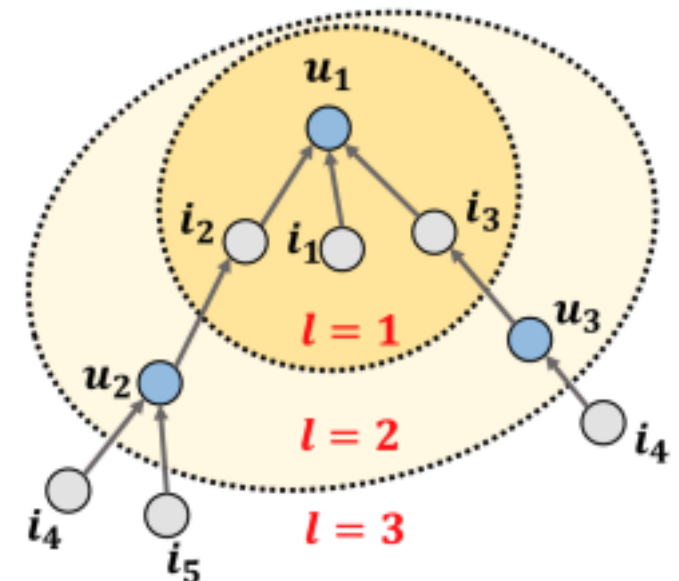
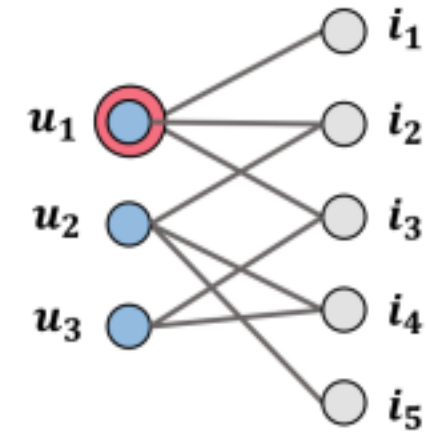
Neural graph collaborative filtering (NGCF)

- Iteratively update node embeddings
- For each user, u , construct message from each connected item, v as

$$\mathbf{m}_{u \leftarrow v} = \frac{1}{\sqrt{|N_u||N_v|}} \left(\mathbf{W}_1 \mathbf{h}_u^{(0)} + \mathbf{W}_2 \left(\mathbf{h}_u^{(0)} \odot \mathbf{h}_v^{(0)} \right) \right)$$

where $\mathbf{W}_1$ and $\mathbf{W}_2$ are learnable weight matrices and $\odot$ is the element wise vector product

- This is similar but distinct from GCN due to interaction term $\mathbf{h}_u^{(0)} \odot \mathbf{h}_v^{(0)}$





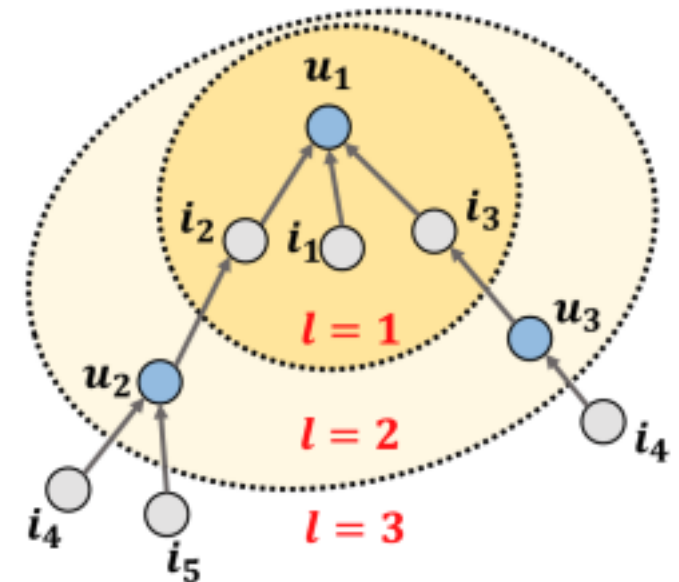
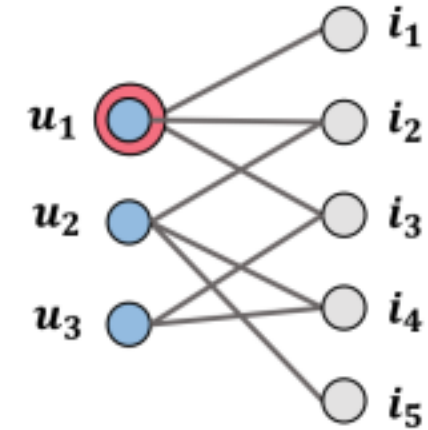
Neural graph collaborative filtering (NGCF)

- Iteratively update node embeddings
- For each user, u , construct message from each connected item, v as

$$\mathbf{m}_{u \leftarrow v} = \frac{1}{\sqrt{|N_u||N_v|}} \left(\mathbf{W}_1 \mathbf{h}_u^{(0)} + \mathbf{W}_2 \left(\mathbf{h}_u^{(0)} \odot \mathbf{h}_v^{(0)} \right) \right)$$

- Aggregate messages from neighbor items

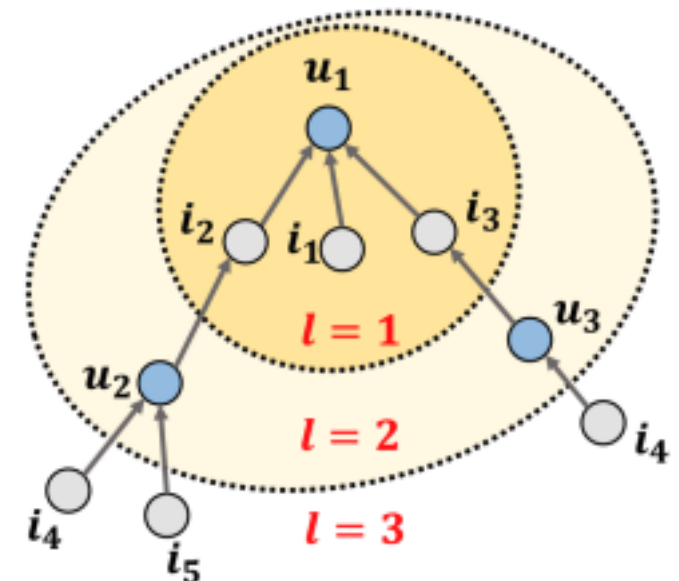
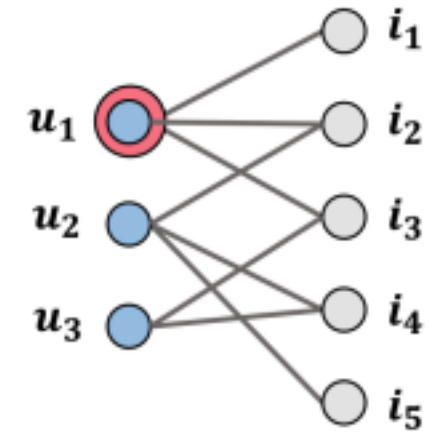
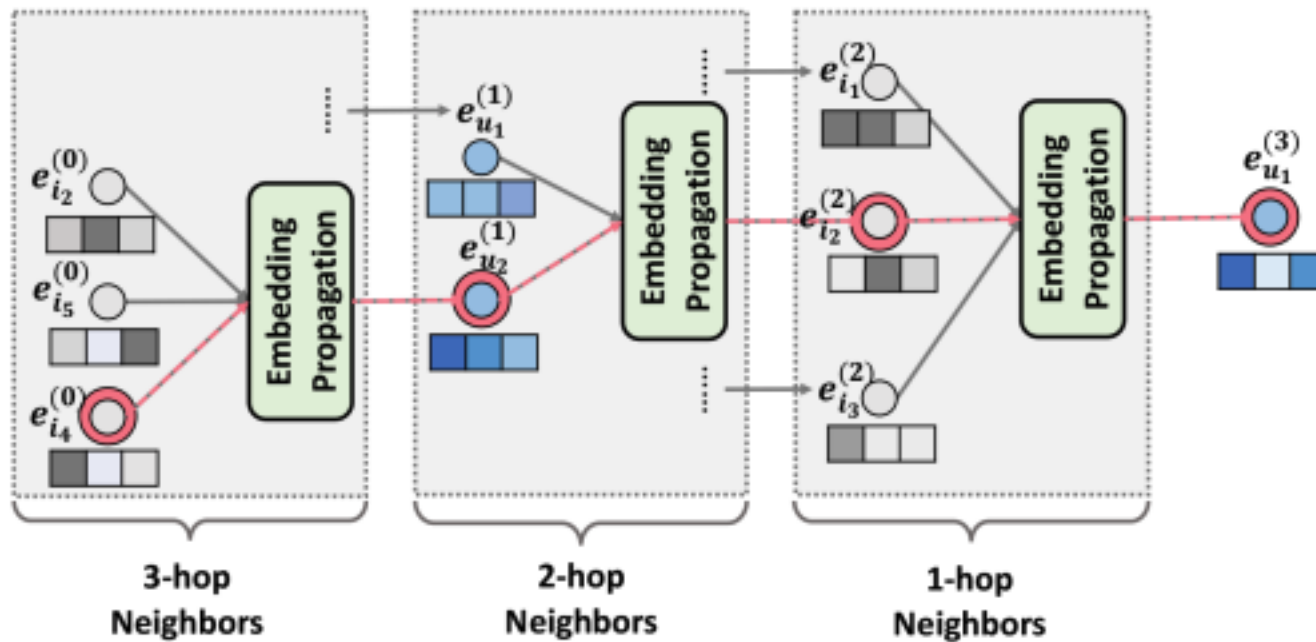
$$\mathbf{h}_u^{(1)} = \text{LeakyReLU} \left(\mathbf{m}_{u \leftarrow u} + \sum_{v \in N_u} \mathbf{m}_{u \leftarrow v} \right)$$



Neural graph collaborative filtering (NGCF)

- Higher level-interactions from multiple layers

$$h_u^{(l)} = \text{LeakyReLU} \left(m_{u \leftarrow u}^{(l)} + \sum_{v \in N_u} m_{u \leftarrow v}^{(l)} \right)$$

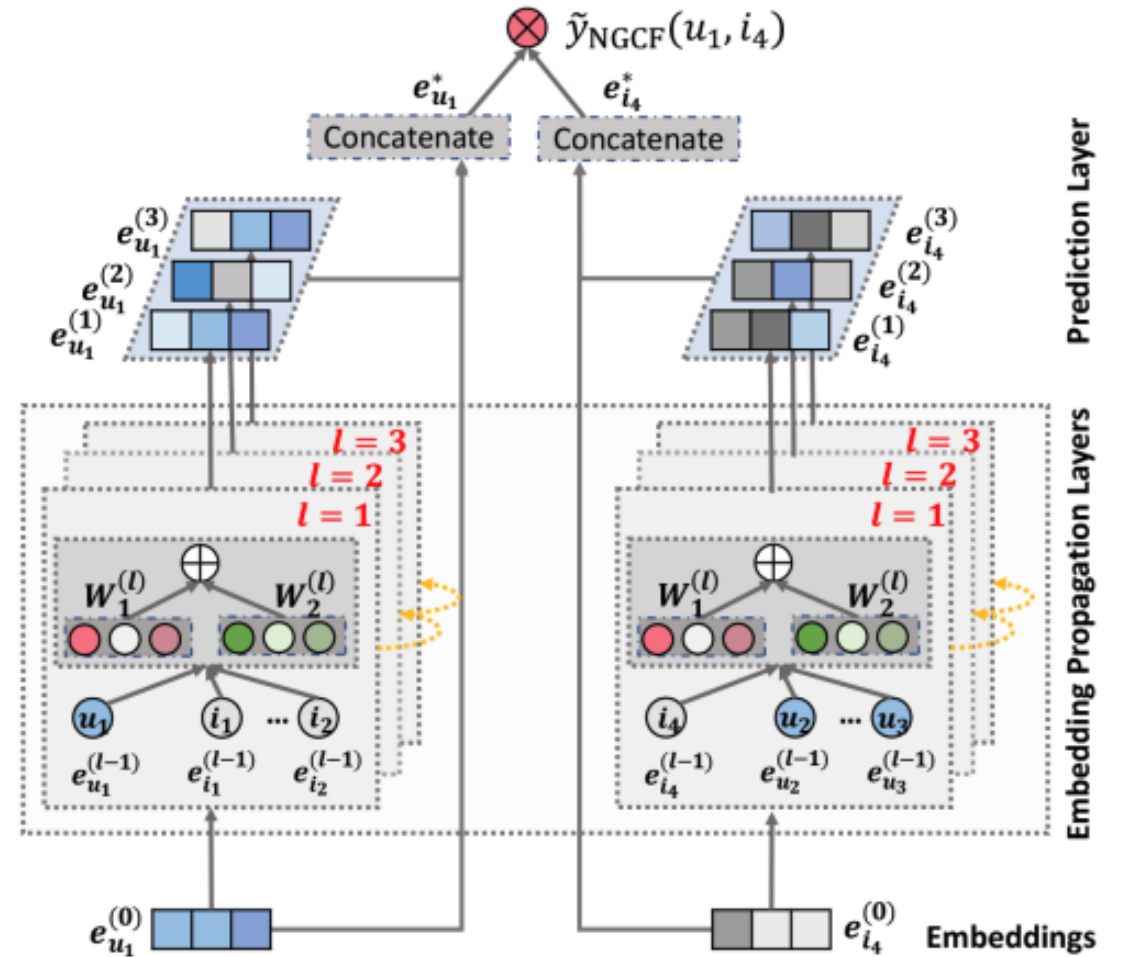


Neural graph collaborative filtering (NGCF)

- The same procedure is used to pass messages and update the item embeddings

$$\mathbf{m}_{v \leftarrow u} = \frac{1}{\sqrt{|N_u||N_v|}} \left(\mathbf{W}_1 \mathbf{h}_v^{(0)} + \mathbf{W}_2 \left(\mathbf{h}_u^{(0)} \odot \mathbf{h}_v^{(0)} \right) \right)$$

$$\mathbf{h}_v^{(l)} = \text{LeakyReLU} \left(\mathbf{m}_{v \leftarrow v}^{(l)} + \sum_{u \in N_v} \mathbf{m}_{v \leftarrow u}^{(l)} \right)$$





LightGCN





Is the complexity of NGCF necessary?

- NGCF uses feature updates and non-linear transformations via W_1 , W_2 and *LeakyReLU*
- Noting that the input “features” are just randomly initialized user and item embeddings, do we need this feature updates?
- NO!
- Let’s examine LightGCN



Light graph convolution network (LightGCN)

- Given: user-item bipartite graph
- LightGCN framework
 - Randomly initialize embeddings for each user & item node (same as NGCF)
- Aggregate messages without transformations!

$$\mathbf{h}_u^{(l+1)} = \frac{1}{\sqrt{|N_u||N_v|}} \sum_{v \in N_u} \mathbf{h}_v^l \quad \text{User embeddings}$$

$$\mathbf{h}_v^{(l+1)} = \frac{1}{\sqrt{|N_u||N_v|}} \sum_{u \in N_v} \mathbf{h}_u^l \quad \text{Item embeddings}$$

Light graph convolution network (LightGCN)

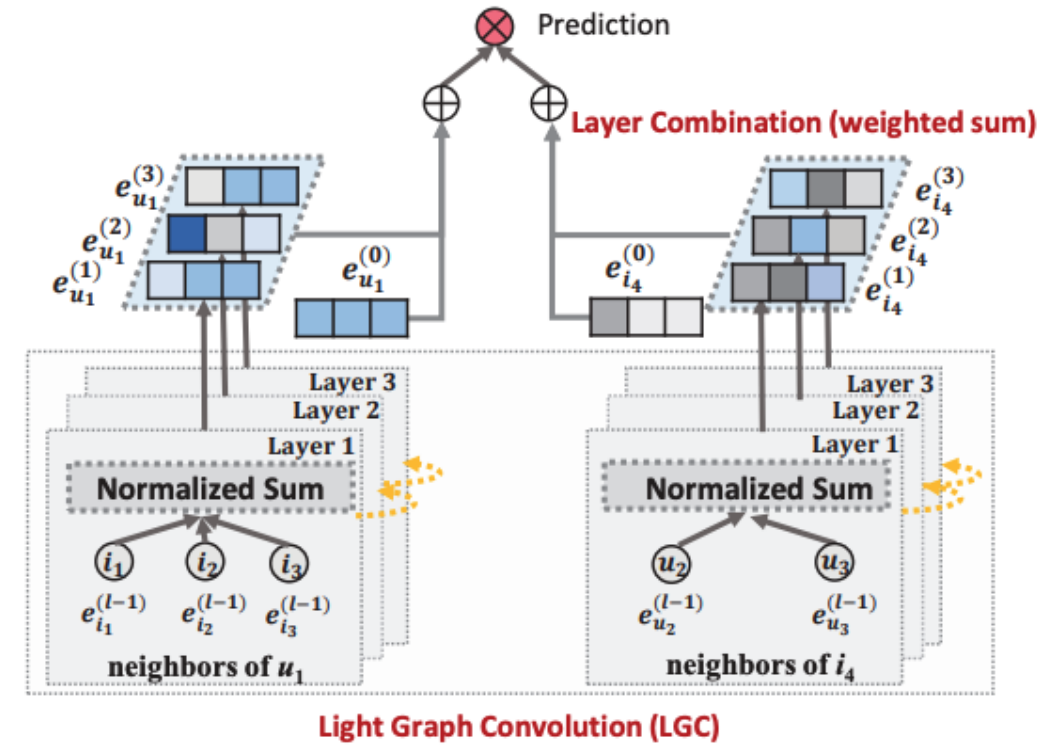
- After L layers, the final embeddings are

$$\mathbf{h}_u = \sum_{l=0}^L \alpha_l \mathbf{h}_u^{(l)} \quad \text{User embeddings}$$

$$\mathbf{h}_v = \sum_{l=0}^L \alpha_l \mathbf{h}_v^{(l)} \quad \text{Item embeddings}$$

where α_l are scaling parameters that can be learned though in LightGCN they are constants $\frac{1}{L+1}$

- Only trainable parameters are the initial embeddings $\mathbf{h}_u^{(0)}$ and $\mathbf{h}_v^{(0)}$





Light graph convolution network (LightGCN)

- Only trainable parameters are the initial embeddings $\mathbf{h}_u^{(0)}$ and $\mathbf{h}_v^{(0)}$
- Do we need to propagate the embeddings at all?

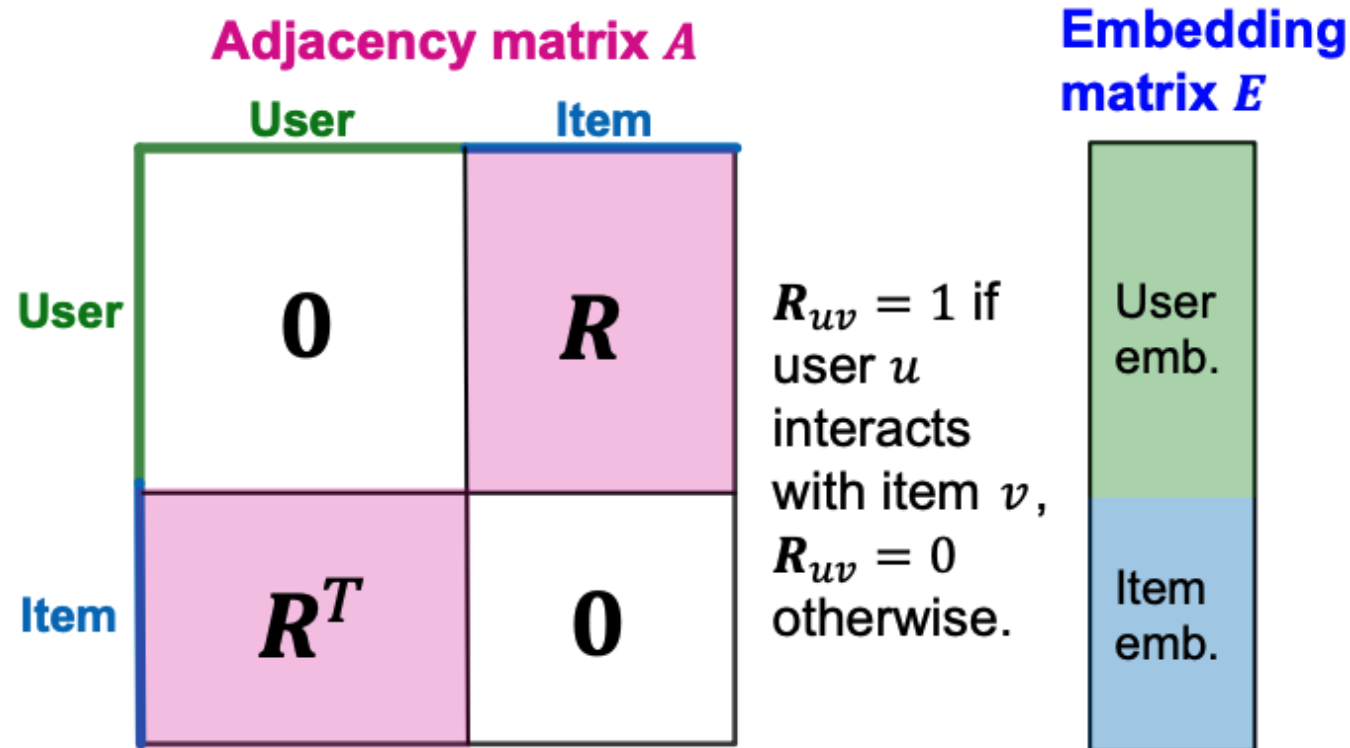
$$\mathbf{h}_u^{(l+1)} = \frac{1}{\sqrt{|N_u||N_v|}} \sum_{v \in N_u} \mathbf{h}_v^l$$
$$\mathbf{h}_v^{(l+1)} = \frac{1}{\sqrt{|N_u||N_v|}} \sum_{u \in N_v} \mathbf{h}_u^l$$

- Yes! Let's find out why by considering the matrix form.



Light graph convolution network (LightGCN)

- We can write the adjacency matrix of an undirected bipartite graph as a block antidiagonal matrix
- We can combine the initial (random) user and item embeddings in a single matrix





Light graph convolution network (LightGCN)

- We can write the adjacency matrix of an undirected bipartite graph as a block antidiagonal matrix
- We can combine the initial (random) user and item embeddings in a single matrix
- The LightGCN embedding updates in matrix form are given by

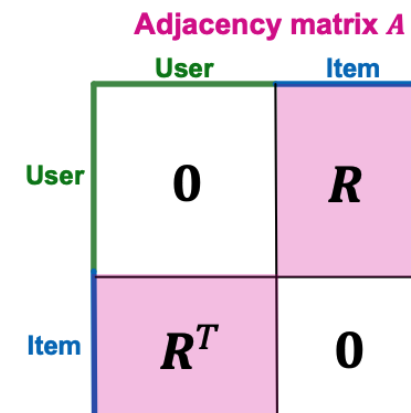
$$\mathbf{H}^{(l+1)} = (\mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}) \mathbf{H}^{(l)}$$

where $\tilde{\mathbf{A}} = \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$ and $\mathbf{D}$ is the diagonal degree matrix

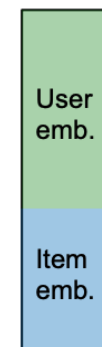
- The final embeddings are given by

$$\begin{aligned} \mathbf{E} &= a_0 \mathbf{H}^{(0)} + a_1 \mathbf{H}^{(1)} + a_2 \mathbf{H}^{(2)} + \dots + a_L \mathbf{H}^{(L)} \\ &= a_0 \mathbf{H}^{(0)} + a_1 \tilde{\mathbf{A}} \mathbf{H}^{(0)} + a_2 \tilde{\mathbf{A}}^2 \mathbf{H}^{(0)} \dots + a_L \tilde{\mathbf{A}}^L \mathbf{H}^{(0)} \end{aligned}$$

α_l are scaling parameters that can be learned
though in LightGCN they are constants $\frac{1}{L+1}$



Embedding matrix $\mathbf{E}$





Light graph convolution network (LightGCN)

- The final embeddings are given by

$$\begin{aligned} E &= a_0 \mathbf{H}^{(0)} + a_1 \mathbf{H}^{(1)} + a_2 \mathbf{H}^{(2)} + \dots + a_L \mathbf{H}^{(L)} \\ &= a_0 \mathbf{H}^{(0)} + a_1 \tilde{\mathbf{A}} \mathbf{H}^{(0)} + a_2 \tilde{\mathbf{A}}^2 \mathbf{H}^{(0)} \dots + a_L \tilde{\mathbf{A}}^L \mathbf{H}^{(0)} \end{aligned}$$

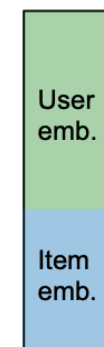
- Each matrix multiplication **diffuses the current embedding to its one-hop neighbors**
 - Embeddings at **different layers capture different semantics**
 - First layer enforces similarity of directly connected users / items
 - Second layer enforces some similarity of users (items) overlap on item (user) interactions
 - Use summation of layer embeddings to address over-smoothing

α_l are scaling parameters that can be learned
though in LightGCN they are constants $\frac{1}{L+1}$

Adjacency matrix A

	User	Item
User	0	R
Item	R^T	0

Embedding matrix E





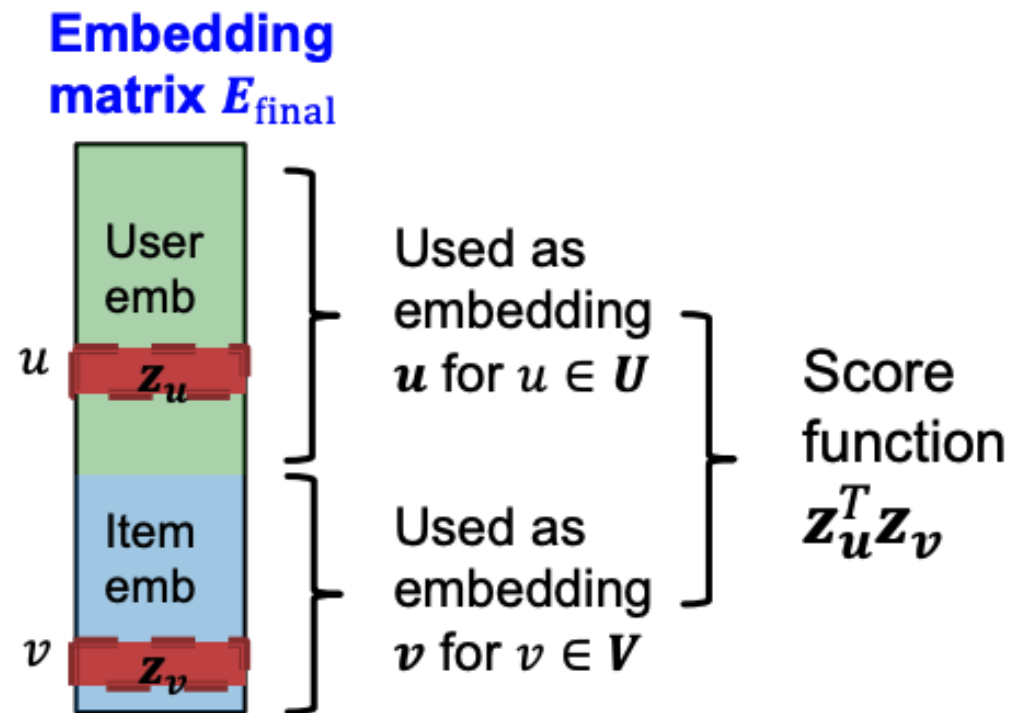
Light graph convolution network (LightGCN)

- The final embeddings are given by

$$\begin{aligned} E &= a_0 \mathbf{H}^{(0)} + a_1 \mathbf{H}^{(1)} + a_2 \mathbf{H}^{(2)} + \dots + a_L \mathbf{H}^{(L)} \\ &= a_0 \mathbf{H}^{(0)} + a_1 \tilde{\mathbf{A}} \mathbf{H}^{(0)} + a_2 \tilde{\mathbf{A}}^2 \mathbf{H}^{(0)} \dots + a_L \tilde{\mathbf{A}}^L \mathbf{H}^{(0)} \end{aligned}$$

- Score function based on the final embeddings E

α_l are scaling parameters that can be learned though in LightGCN they are constants $\frac{1}{L+1}$





Model training

- Our training objective (recall@K) is not differentiable
 - Cannot apply gradient-based optimization
- Two alternative loss functions are used to address this
 - Binary loss
 - Bayesian personalized rankings (BPR) loss
- Both alternatives align well with the original training objective though BPR is typically preferred (let's see why)



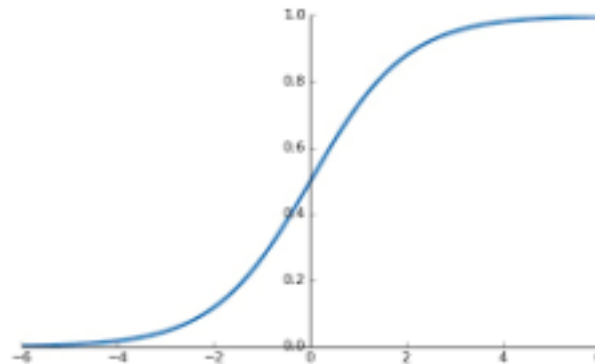
NGCF & LightGCN Training

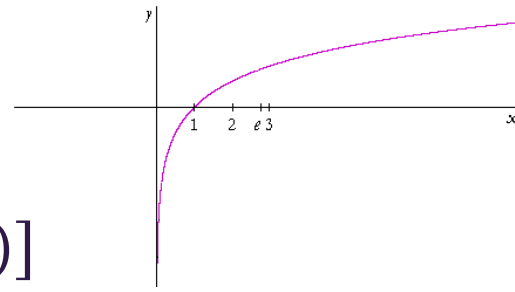




Binary loss

- Define positive/negative edges
 - A set of **positive edges** $\mathbf{E}$ (i.e., observed user-item interactions)
 - A set of **negative edges** $\mathbf{E}_{neg}\{(u, v) | (u, v) \notin \mathbf{E}, u \in \mathbf{U}, v \in \mathbf{V}\}$
- Recall sigmoid function $\sigma(z) = \frac{1}{1+e^{-x}}$
 - Maps all real values to $[0, 1]$





Binary loss

- Binary classification of **positive**/negative edges using $\sigma[f_\theta(\mathbf{u}, \mathbf{v})]$

$$\mathcal{L}(\mathbf{E}, \mathbf{E}_{neg}) = -\frac{1}{|\mathbf{E}|} \sum_{(u,v) \in \mathbf{E}} \log[\sigma[f_\theta(\mathbf{u}, \mathbf{v})]] - \frac{1}{|\mathbf{E}_{neg}|} \sum_{(u',v') \in \mathbf{E}_{neg}} \log[1 - \sigma[f_\theta(\mathbf{u}', \mathbf{v}')]]$$

Want f_θ large and positive so that σ is close to 1 and log is close to zero, otherwise the log will be negative and the first term will be positive

Want f_θ large and negative so that σ is close to 0 and log is close to zero, otherwise the log will be negative and the second term will be positive

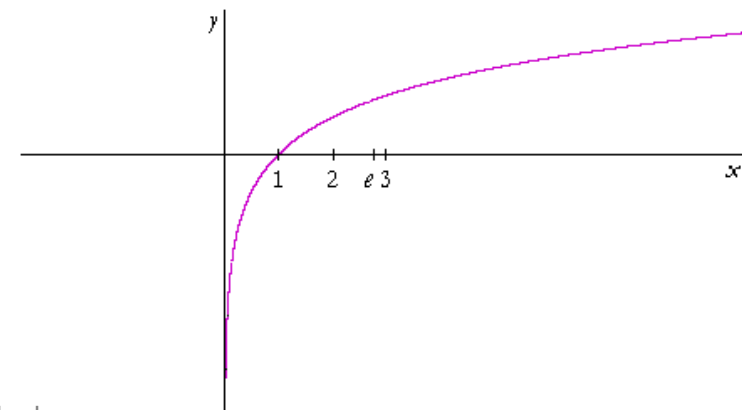
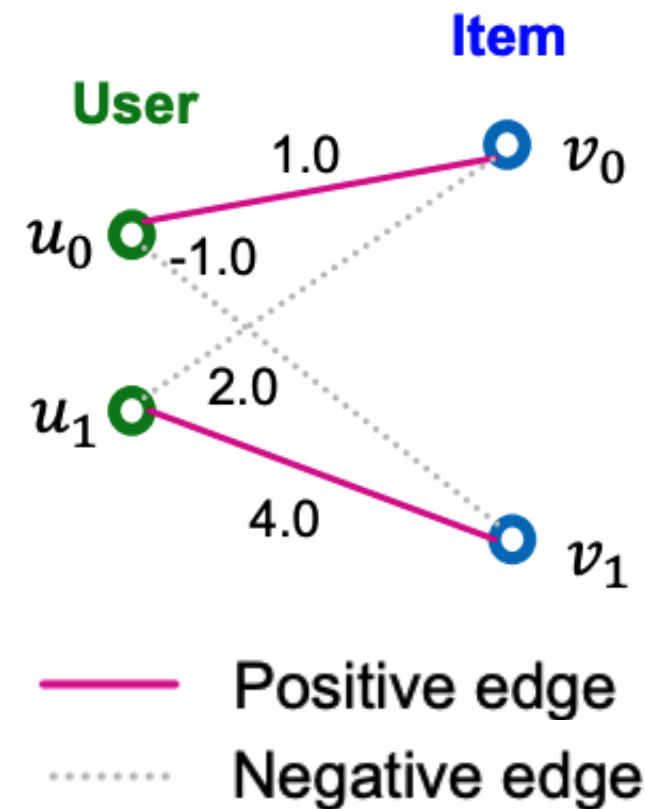
- Minibatches used in practice
- Binary loss pushes the scores of **positive** edges higher than those of negative edges
- Issue**: the scores of **all** positive edges are pushed higher than those of **all** negative edges
 - Unnecessarily penalizes model predictions even if the Recall@K metric is perfect
 - Why? (next slide)



Binary loss

- Consider simplest case:
 - Two users, two items
 - Metric: Recall@1
 - Assume model has assigned edge scores as shown
- Training Recall@1 is 1.0 (perfect)
 - v_0 is correctly recommended to u_0
 - v_1 is correctly recommended to u_1
- The binary loss is penalized anyway because the **negative** u_1, v_0 edge has a positive score that adds to the loss function

$$-\frac{1}{|\mathbf{E}_{neg}|} \sum_{(u', v') \in \mathbf{E}_{neg}} \log[1 - \sigma[f_{\theta}(\mathbf{u}', \mathbf{v}')]]$$





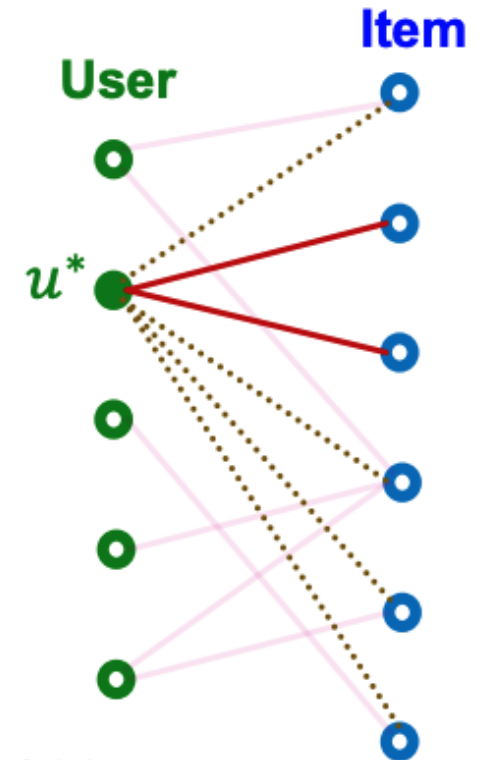
Binary loss

- The binary loss is not personalized in the sense that the positive/negative edges are considered across all users simultaneously
- The Recall@K metric is inherently personalized
 - The binary loss is overly stringent for personalized recommendation
 - Bayesian personalized ranking (BPR) loss will address this



Bayesian personalized ranking (BPR) loss

- For each user $u^* \in U$, define the rooted positive/negative edges as
 - Positive edges rooted at u^*
 - $E(u^*) \equiv \{(u^*, v) | (u^*, v) \in E\}$
 - Negative edges rooted at u^*
 - $E_{neg}(u^*) \equiv \{(u^*, v) | (u^*, v) \in E_{neg}\}$





Bayesian personalized ranking (BPR) loss

- For each user $u^* \in U$, we want the scores of rooted positive edges, $E(u^*)$, to be higher than those of rooted negative edges, $E_{neg}(u^*)$

- BPR loss function for user u^*

$$\mathcal{L}_{bpr}(u^*) = \frac{1}{|E(u^*)||E_{neg}(u^*)|} \sum_{(u^*, v) \in E(u^*)} \sum_{(u^*, v') \in E_{neg}(u^*)} -\log[\sigma(f_\theta(u^*, v) - f_\theta(u^*, v'))]$$

- Overall model loss

$$\frac{1}{|U|} \sum_{u^* \in U} \mathcal{L}_{bpr}(u^*)$$

Mini-batch training with BPR loss

- In each mini-batch, sample a subset of users $U^* \subset U$
 - For each user $u^* \in U^*$, sample one positive item v_{pos} and a set of negative items V_{neg}
 - The mini-batch loss is then

$$\frac{1}{|U^*||V_{neg}|} \sum_{u^* \in U^*} \sum_{v_{neg} \in V_{neg}} -\log \left[\sigma \left(f_{\theta}(u^*, v_{pos}) - f_{\theta}(u^*, v_{neg}) \right) \right]$$

- This loss function is used for both NGCF and LightGCN
- Recall, the initial node and item embeddings for NGCF and LightGCN are trainable parameters.
- During training with PyTorch (or other autograd library), it is necessary to calculate gradients for these parameters!!

